

Name: Om Dhamame

PRN- 23070122155

DevOps Lab Assessment Questions

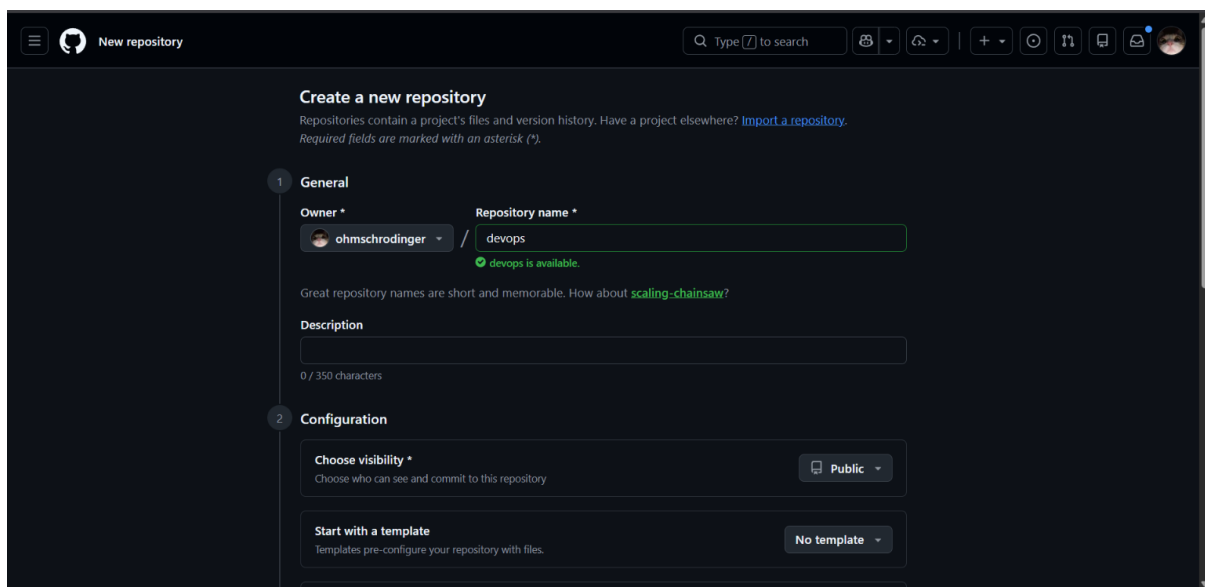
1. TW1 - Continuous Evaluation: Foundational DevOps Skills (10 Marks)

Instructions: Complete the following tasks in your lab environment. Document your commands and outputs as evidence.

Scenario: You are a developer working on a new feature for a web application.

1. Git Workflow & Collaboration (4 Marks)

- **Task 1.1:** Initialize a new Git repository for a simple "Hello World" Python Flask application (provided to you). Add the initial code and commit it to the main branch. (1 Mark)



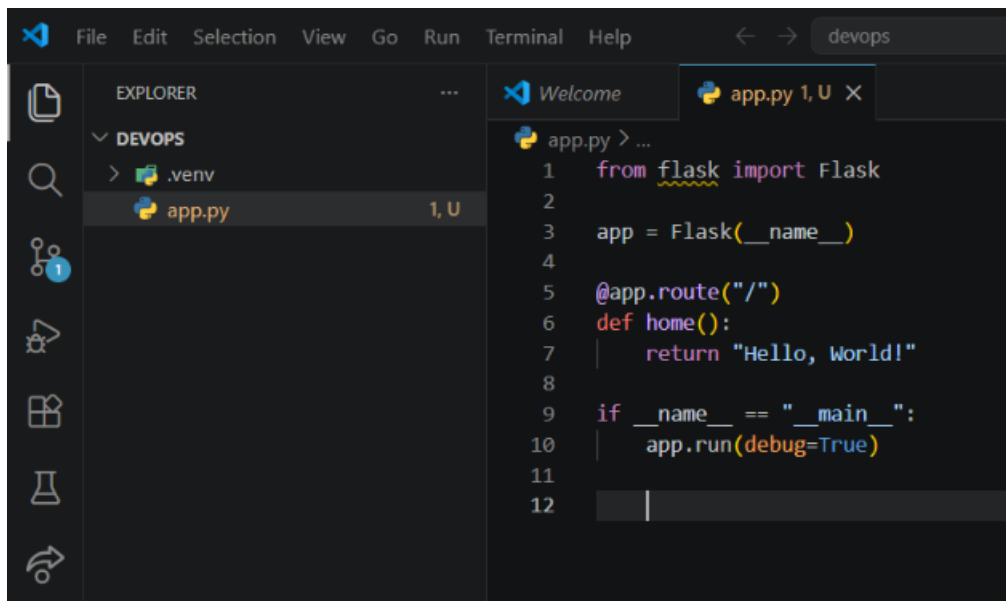
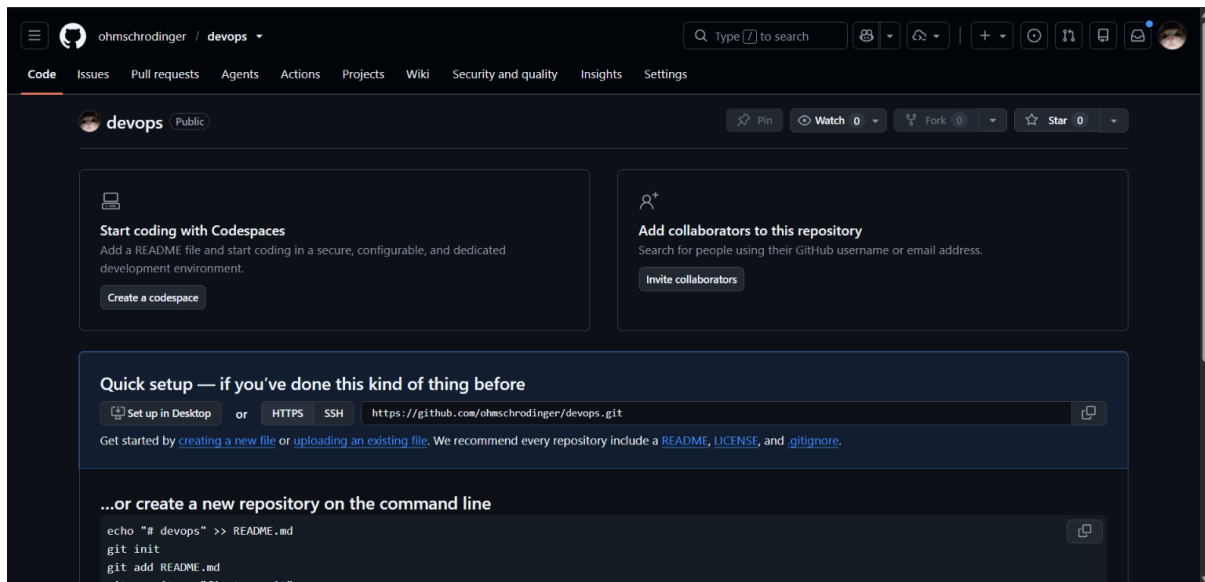
The screenshot shows the GitHub 'Create a new repository' page. The page is titled 'Create a new repository' and includes a sub-header 'Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository.](#)'. Below this, there are two main sections: 'General' and 'Configuration'.

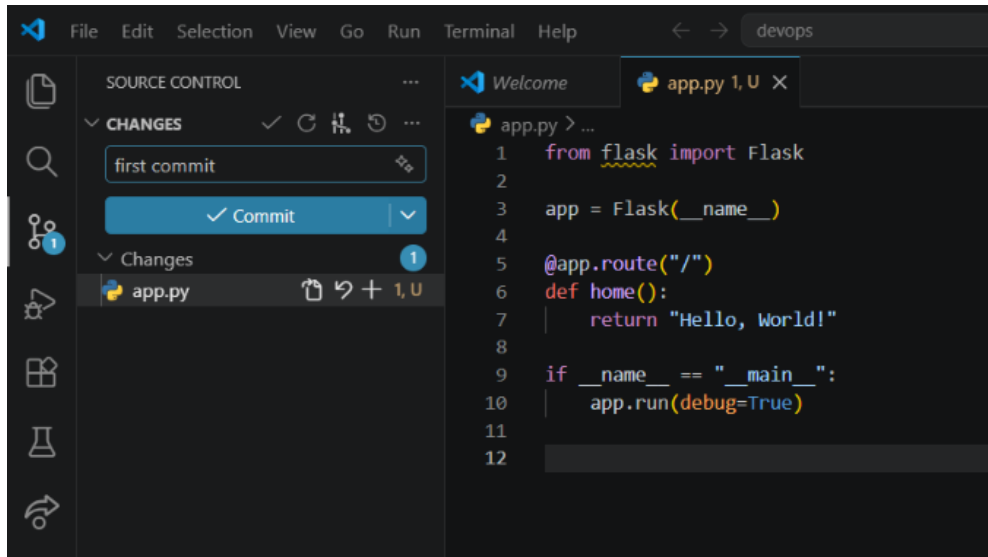
General Section:

- Owner ***: A dropdown menu showing 'ohmschrodinger'.
- Repository name ***: A text input field containing 'devops'. A green checkmark and the text 'devops is available.' are displayed below the field.
- Description**: A text input field with a placeholder '0 / 350 characters'.

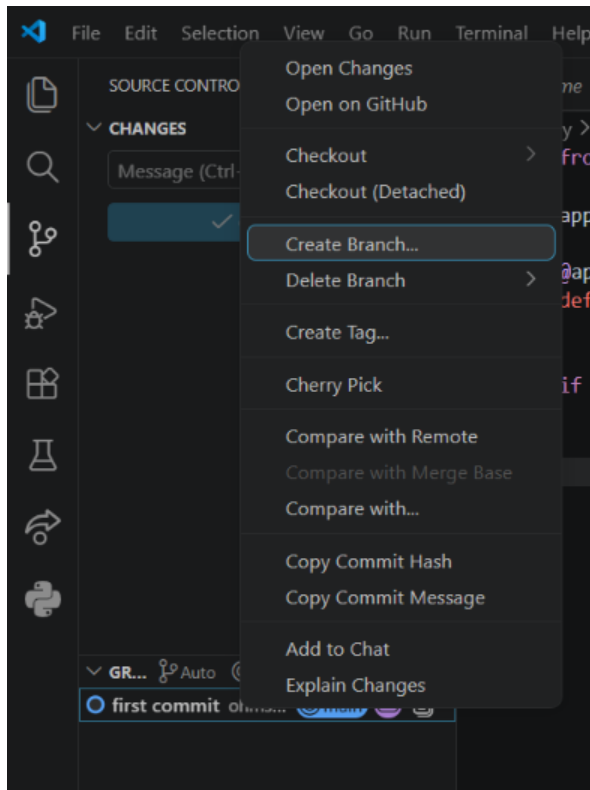
Configuration Section:

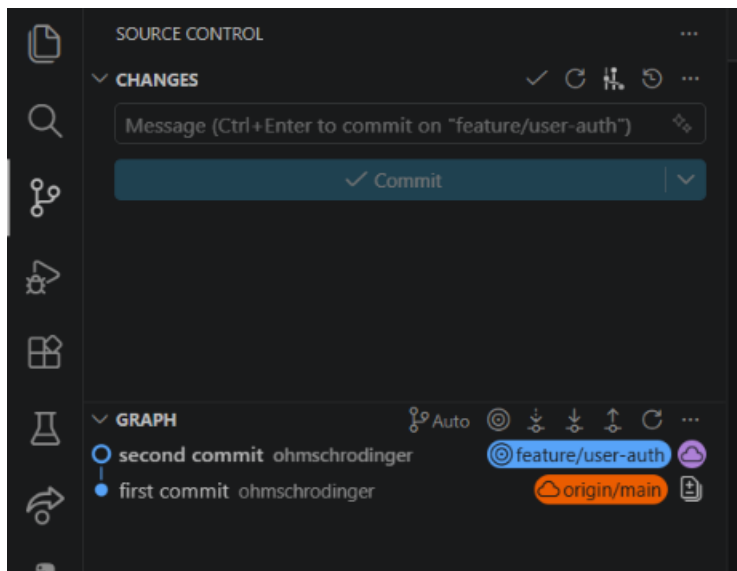
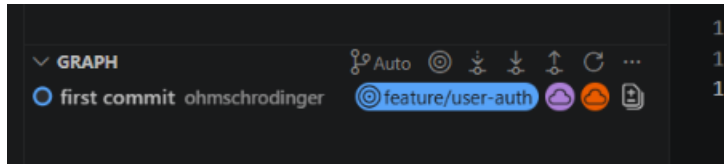
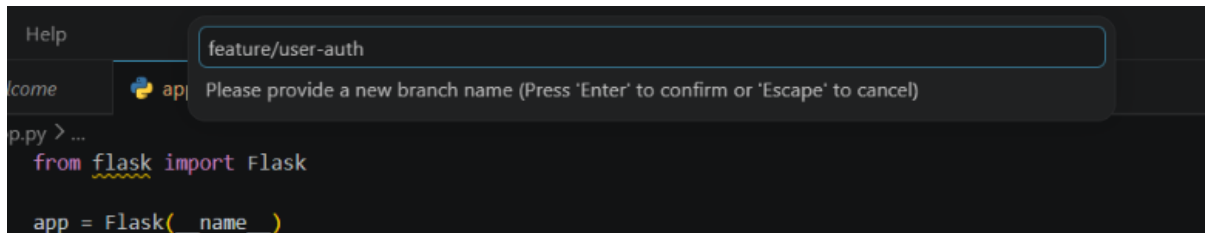
- Choose visibility ***: A dropdown menu showing 'Public'.
- Start with a template**: A dropdown menu showing 'No template'.



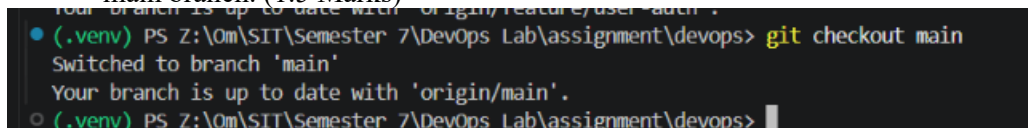


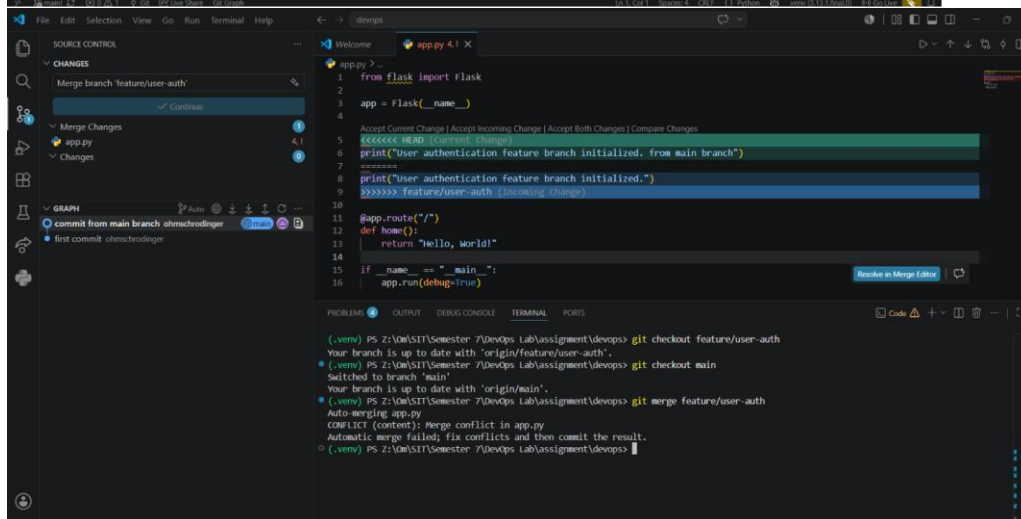
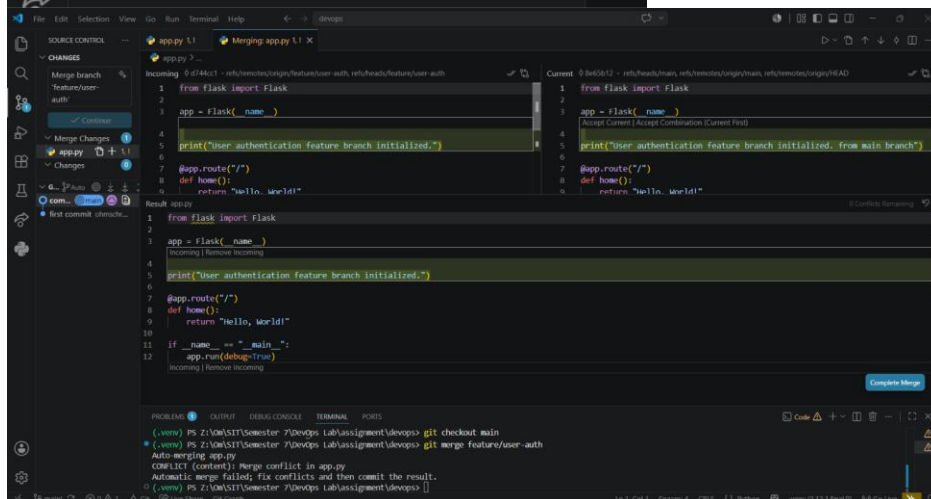
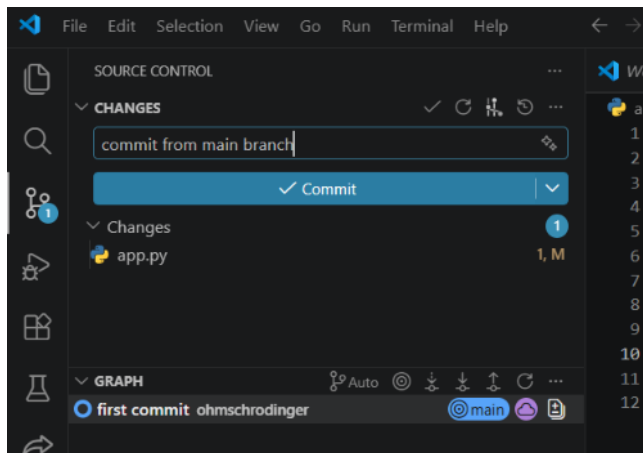
- **Task 1.2:** Create a new branch named `feature/user-auth`. Make a small modification (e.g., add a new print statement) in this branch, commit the changes, and push the branch to a remote repository (e.g., GitHub/GitLab). (1.5 Marks)

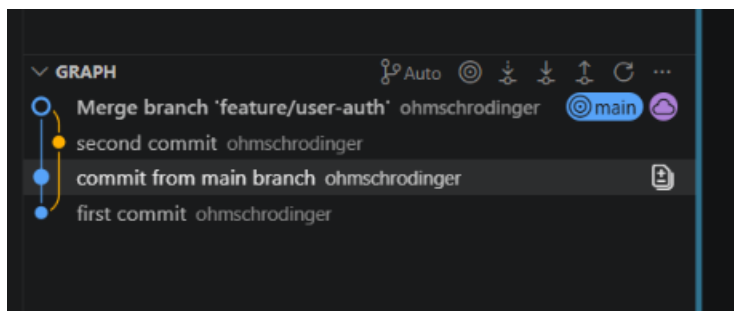
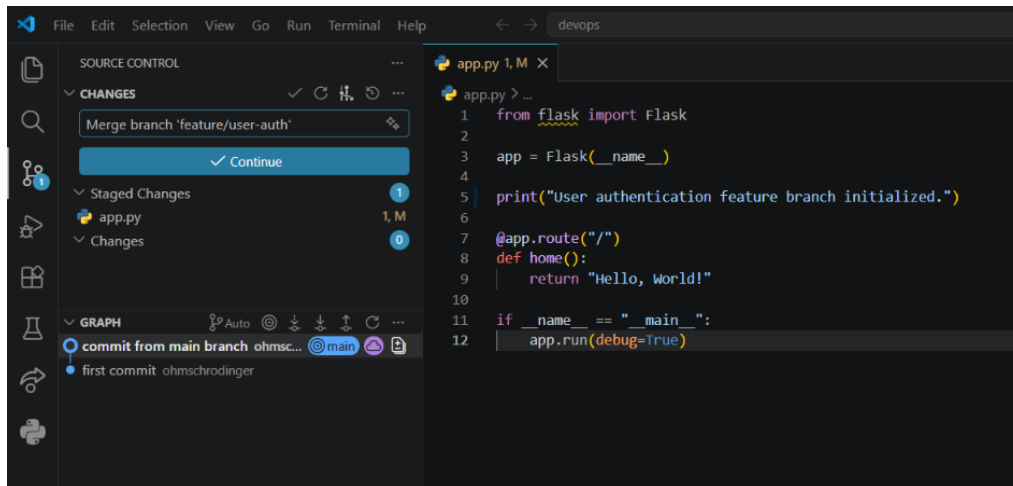




- **Task 1.3:** Simulate a conflict: On the main branch, modify the *same line* you changed in feature/user-auth. Commit this change. Now, attempt to merge feature/user-auth into main. Resolve the merge conflict manually and commit the resolution. Push the updated main branch. (1.5 Marks)

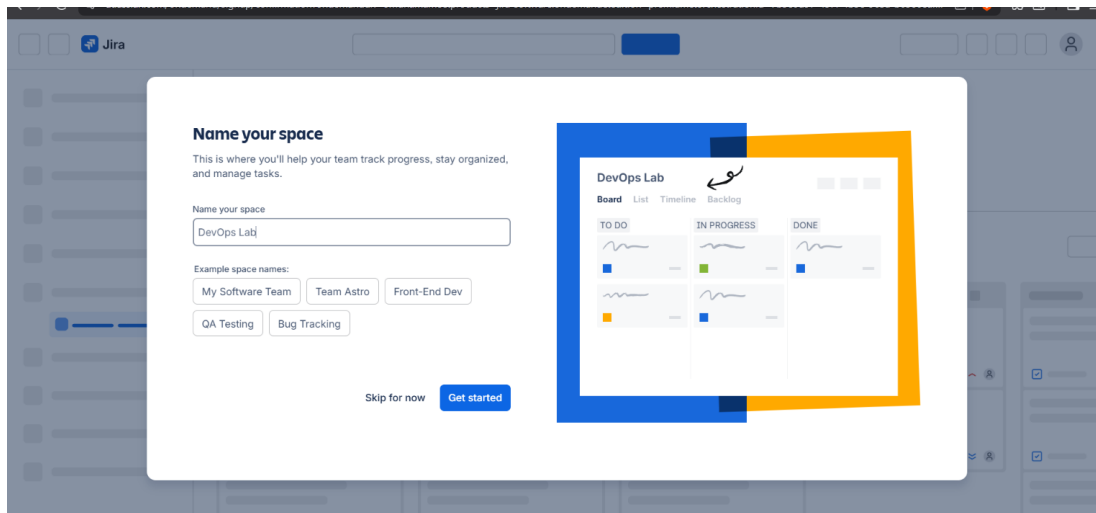


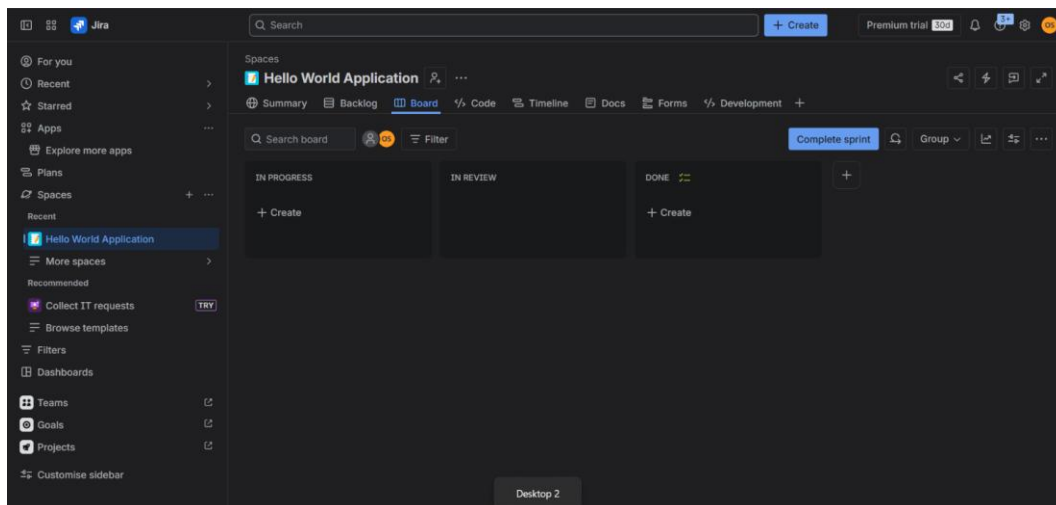




2. Jira Project & Issue Tracking (3 Marks)

- **Task 2.1:** Log in to your free Jira Cloud instance. Create a new "Scrum" project for the "Hello World" application. (1 Mark)





- **Task 2.2:** Create the following issues within your new project:
 - One "Story" titled "Implement User Authentication Feature".

Create Story

Required fields are marked with an asterisk *

Space *

Work type *

[Learn about work types](#)

Status

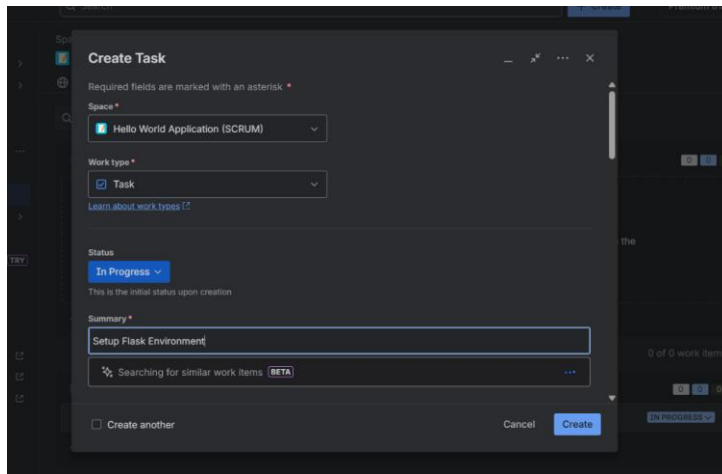
This is the initial status upon creation

Summary *

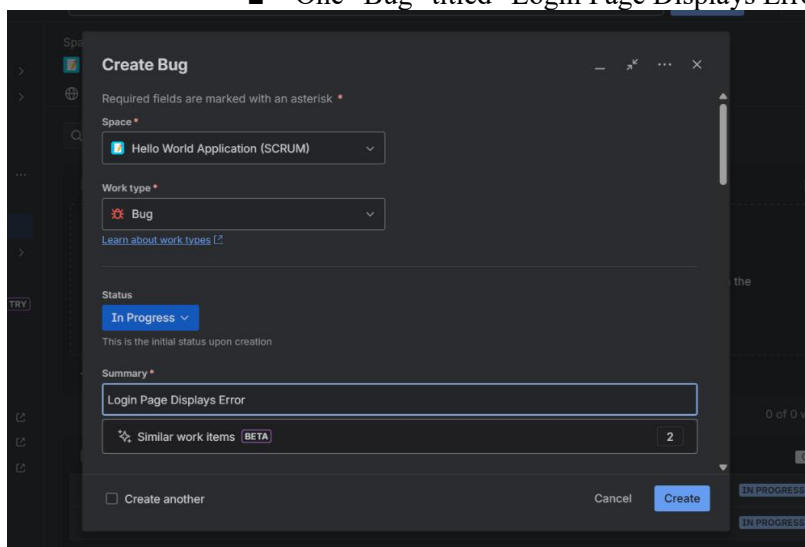
Searching for similar work items **BETA** [...](#)

☐ Create another Cancel Create

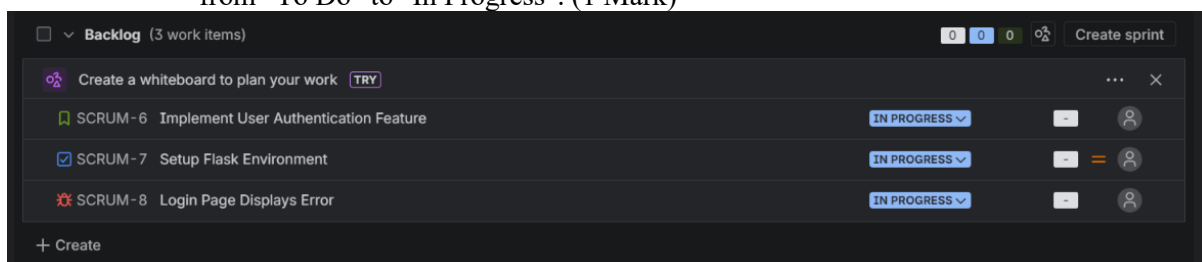
- One "Task" titled "Setup Flask Environment".



- One "Bug" titled "Login Page Displays Error". (1 Mark)

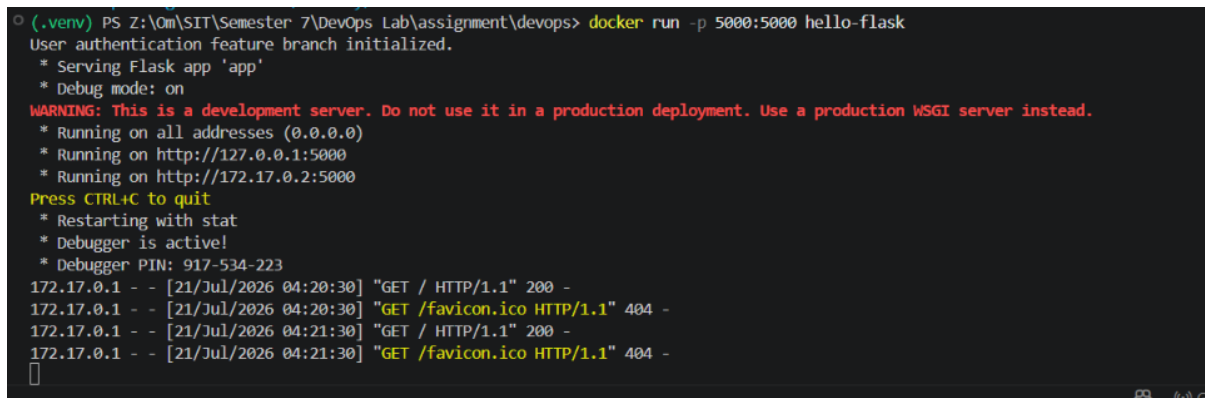
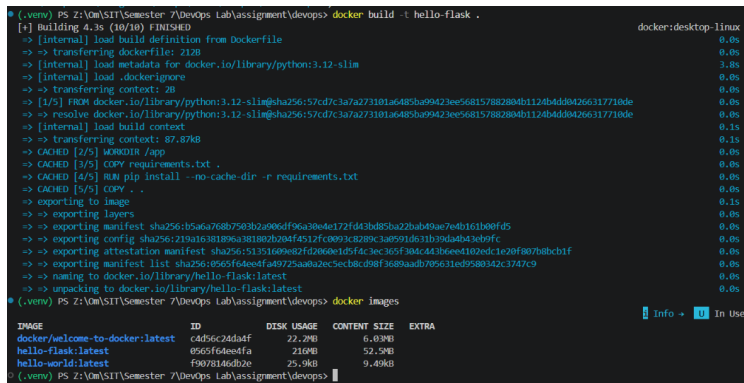
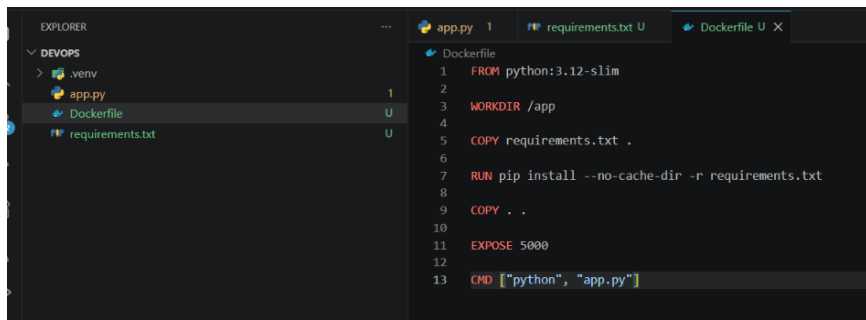


- **Task 2.3:** Navigate to the project's board. Move the "Setup Flask Environment" task from "To Do" to "In Progress". (1 Mark)



3. Basic Containerization (Docker) & Jenkins Freestyle Project (3 Marks)

- **Task 3.1:** Create a Dockerfile for your Python Flask "Hello World" application. The Dockerfile should build a minimal image that runs the application on port 5000. Build the Docker image locally and verify it runs correctly. (1.5 Marks)



- **Task 3.2:** Set up a Jenkins Freestyle project. Configure it to pull your Git repository (from Task 1.1) and execute a build step that lists the contents of the workspace. Trigger a build and provide a screenshot of the successful build output. (1.5 Marks)

New Item

Enter an item name

Hello-Flask

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.



Multibranch Pipeline

OK

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

☐ Inspect build log for published build scans☐ Terminate a build if it's stuck☐ With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute Windows batch command ?

Command

See the list of available environment variables

dir

Advanced ▾

+ Add build step

Save

Apply

- ▶ Build Now
- 🗑 Delete Project
- ⚙ Configure
- ✎ Rename

- Last build (#3), 6 min 50 sec ago
- Last stable build (#3), 6 min 50 sec ago
- Last successful build (#3), 6 min 50 sec ago
- Last failed build (#2), 8 min 12 sec ago
- Last unsuccessful build (#2), 8 min 12 sec ago
- Last completed build (#3), 6 min 50 sec ago

Builds > ...

Today

✓ #3 10:21

✗ #2 10:19

✗ #1 10:17

✓ Build scheduled

Jenkins / Hello-Flask / #3

🔍 ⚙

Status

</> Changes

📄 Console Output

🗑 Delete build '#3'

🔗 Git Build Data

✎ Edit build information

← Previous Build

✓ #3 (21 Jul 2026, 10:21:12)

Started by user Ohm Schrodinger

git

Revision: 56a864e0f7a739b722837b1502e5850ffc5c7f3
Repository: <https://github.com/ohmschrodinger/devops>

- refs/remotes/origin/main

</> No changes.

Add description

Keep this build for

Started 8.7 s
Took 5 sec

Jenkins / Hello-Flask / #3

🔍 ⚙ 👤

previous build

Fetching upstream changes from <https://github.com/ohmschrodinger/devops>

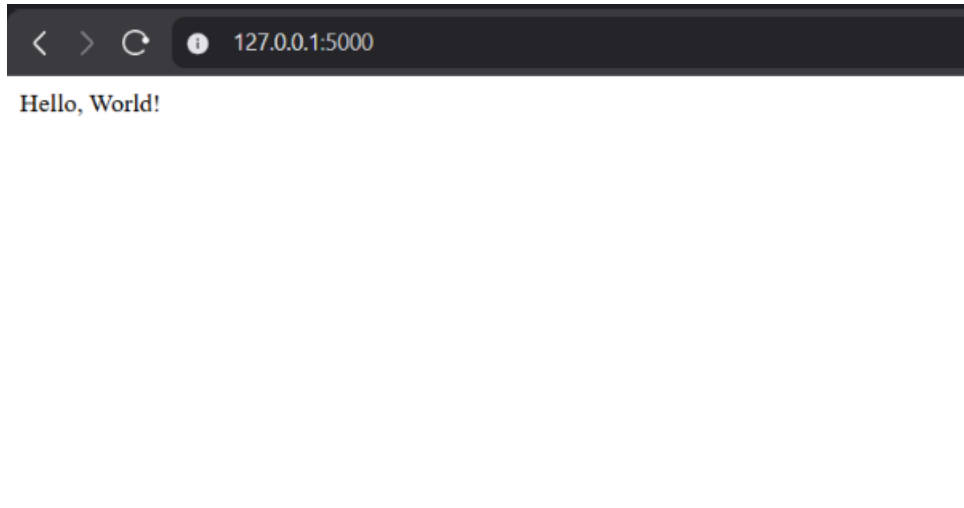
```
> git.exe --version # timeout=10
> git --version # 'git version 2.48.1.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/ohmschrodinger/devops +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 56a864e0f7a739b722837b1502e5850ffc5c7f3 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 56a864e0f7a739b722837b1502e5850ffc5c7f3 # timeout=10
Commit message: "push"
First time build. Skipping changelog.
[Hello-Flask] $ cmd /c call C:\WINDOWS\TEMP\jenkins12172956882412754514.bat

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Flask>dir
Volume in drive C is OS
Volume Serial Number is A661-A685

Directory of C:\ProgramData\Jenkins\jenkins\workspace\Hello-Flask

21-07-2026 10:21 <DIR>      .
21-07-2026 10:17 <DIR>      ..
21-07-2026 10:21          250 app.py
21-07-2026 10:21          173 Dockerfile
21-07-2026 10:21           5 requirements.txt
                3 File(s)      428 bytes
                2 Dir(s)    49,181,671,424 bytes free

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Flask>exit 0
Finished: SUCCESS
```



2. TW2 - Continuous Evaluation: Advanced DevOps Practices (10 Marks)

Instructions: Build upon your previous work. Document your commands, configuration files (e.g., docker-compose.yml, Kubernetes YAMLs), and outputs.

Scenario: You need to enhance your "Hello World" application to include a database and deploy it using orchestration tools.

1. Docker Compose for Multi-Container App (3 Marks)

- **Task 1.1:** Modify your "Hello World" Flask application to include a simple PostgreSQL database (e.g., the app connects to and stores a single entry). (1 Mark)

```
app.py 2. M X requirements.txt M Dockerfile docker-compose.yml U
app.py > home
1  from flask import Flask
2  import psycopg2
3  import time
4
5  app = Flask(__name__)
6
7  print("User authentication feature branch initialized.")
8
9
10 def get_db_connection():
11     while True:
12         try:
13             conn = psycopg2.connect(
14                 host="db",
15                 database="hello_db",
16                 user="postgres",
17                 password="password"
18             )
19             return conn
20         except Exception:
21             print("Database not ready, waiting...")
22             time.sleep(2)
23
24
25
26
27 @app.route("/")
28 def home():
29     conn = get_db_connection()
30
31     cursor = conn.cursor()
32
33     cursor.execute("""
34         CREATE TABLE IF NOT EXISTS messages(
35             id SERIAL PRIMARY KEY,
36             message TEXT
37         )
38     """)
39
40
41     cursor.execute(
42         "INSERT INTO messages(message) VALUES(%s)",
43         ("Hello from Flask + PostgreSQL",)
44     )
45
46
47     conn.commit()
48
49     cursor.close()
50     conn.close()
51
52     return "Hello World! Data stored in PostgreSQL."
53
54
55
```

```
55
56 if __name__ == "__main__":
57
58     app.run(
59         host="0.0.0.0",
60         port=5000,
61         debug=True
62     )
```

- **Task 1.2:** Create a docker-compose.yml file that defines two services: your Flask application and the PostgreSQL database. Configure them to communicate. Bring up the

application using docker-compose up and verify both services are running and communicating. Provide the docker-compose.yml and output. (2 Marks)

```

5 flask-app:
6   build: .
7   container_name: flask-container
8   ports:
9     - "5000:5000"
10  depends_on:
11    - db
12
13
14  db:
15    image: postgres:16
16    container_name: postgres-container
17
18    environment:
19      POSTGRES_USER: postgres
20      POSTGRES_PASSWORD: password
21      POSTGRES_DB: hello_db
22
23    ports:
24      - "5432:5432"

```

```

postgres-container | 2026-07-22 08:11:41.014 UTC [1] LOG:  database system is ready to accept connections
flask-container    | 172.18.0.1 - - [22/Jul/2026 08:11:56] "GET / HTTP/1.1" 200 -
flask-container    | 172.18.0.1 - - [22/Jul/2026 08:11:57] "GET /favicon.ico HTTP/1.1" 404 -
postgres-container | 2026-07-22 08:12:49.363 UTC [77] ERROR:  syntax error at or near "Set" at character 2
postgres-container | 2026-07-22 08:12:49.363 UTC [77] STATEMENT:  (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ;
postgres-container | 2026-07-22 08:16:41.082 UTC [62] LOG:  checkpoint starting: time
postgres-container | 2026-07-22 08:16:50.576 UTC [62] LOG:  checkpoint complete: wrote 97 buffers (0.6%); 0 WAL file(s) added, 0 removed, 0 rec
s; distance=426 kB, estimate=426 kB; lsn=0/1989B78, redo lsn=0/1989B40

```

```

PS Z:\Om\SIT\Semester 7\DevOps Lab\assignment\devops> docker exec -it postgres-container psql -U postgres -d hello_db
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

hello_db=# select * from messages;
 id |          message
-----+-----
  1 | Hello from Flask + PostgreSQL
(1 row)

hello_db=#

```

2. Kubernetes Deployment & Service (4 Marks)

- Task 2.1: Ensure your Docker image from TW1 (or an updated one) is accessible (e.g., pushed to a public Docker Hub repository, or Minikube's Docker daemon is used). (1 Mark)

```

PS C:\Users\omdha> minikube start
* minikube v1.38.1 on Microsoft Windows 11 Home Single Language 25H2
* Automatically selected the docker driver
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Downloading Kubernetes v1.35.1 preload ...
  > preloaded-images-k8s-v18-v1...: 272.45 MiB / 272.45 MiB 100.00% 9.66 Mi
  > gcr.io/k8s-minikube/kicbase...: 519.58 MiB / 519.58 MiB 100.00% 7.47 Mi
* Creating docker container (CPUs=2, Memory=4000MB) ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectrl is now configured to use "minikube" cluster and "default" namespace by default

```

```
PS C:\Users\omdha> docker ps
CONTAINER ID   IMAGE                                NAMES      COMMAND                  CREATED        STATUS        PORTS
ebeab3eda438   gcr.io/k8s-minikube/kicbase:v0.0.50 /usr/local/bin/entr... 4 minutes ago Up 4 minutes 127.0.0.1:49965->22/tcp, 127.0.0.1:49963->2376/tcp, 127.0.0.1:49964->5000/tcp, 127.0.0.1:49967->8443/tcp, 127.0.0.1:49966->32443/tcp
11fa41de9dca   devops-flask-app                  flask-container "python app.py"        27 minutes ago Up 27 minutes 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
6ec9bf009dd2   postgres:16                       postgres-container "docker-entrypoint.s..." 27 minutes ago Up 27 minutes 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
```

```
PS C:\Users\omdha> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:49967
CoreDNS is running at https://127.0.0.1:49967/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Users\omdha> kubectl get nodes
NAME        STATUS   ROLES    AGE     VERSION
minikube    Ready   control-plane 5m5s    v1.35.1
PS C:\Users\omdha> docker images
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
devops-flask-app:latest             3bfbaf1d905e                    232MB       57MB          U
docker/welcome-to-docker:latest     c4d56c24da4f                    22.2MB      6.03MB
gcr.io/k8s-minikube/kicbase:v0.0.50 b97074569ae9                    1.94GB      545MB
gcr.io/k8s-minikube/kicbase@sha256:eb4fec00e8ad70adf8e6436f195cc429825ffb85f95afcdb5d8d9deb576f3e93 eb4fec00e8ad                    1.94GB      545MB
hello-flask:latest                  a45ba7fa2d28                    216MB       52.5MB        U
hello-world:latest                  f9078146db2e                    25.9kB      9.49kB
postgres:16                         33f923b05f64                    642MB       166MB         U
PS C:\Users\omdha> minikube image load devops-flask-app:latest
PS C:\Users\omdha>
```

- **Task 2.2:** Create a Kubernetes Deployment YAML for your Flask application. Deploy it to your local Kubernetes cluster (Minikube/Kind). Verify the Pods are running. (1.5 Marks)

```
deployment.yaml > {} spec > {} template > {} spec > {} containers > {} flask-container > {} ports > {} [0] > containerPort
1  apiVersion: apps/v1
2  kind: Deployment
3
4  metadata:
5    name: flask-deployment
6
7  spec:
8    replicas: 2
9
10   selector:
11     matchLabels:
12       app: flask
13
14   template:
15
16     metadata:
17       labels:
18         app: flask
19
20     spec:
21
22       containers:
23
24         - name: flask-container
25
26           image: devops-flask-app:latest
27
28           imagePullPolicy: Never
29
30           ports:
31             - containerPort: 5000
```

```
PS Z:\0m\SIT\Semester 7\DevOps Lab\assignment\devops> kubectl apply -f deployment.yaml
deployment.apps/flask-deployment created
PS Z:\0m\SIT\Semester 7\DevOps Lab\assignment\devops> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
flask-deployment-84964d4d6b-2p662  1/1     Running   0           55s
flask-deployment-84964d4d6b-lbncm  1/1     Running   0           55s
PS Z:\0m\SIT\Semester 7\DevOps Lab\assignment\devops> _
```

- **Task 2.3:** Create a Kubernetes Service YAML (type NodePort or LoadBalancer if supported) to expose your Flask application. Apply the manifest and provide the command to access your application from your host machine. (1.5 Marks)

```
service.yaml > {} spec > {} ports > {} [0] > nodePort
1  apiVersion: v1
2  kind: Service
3
4  metadata:
5    name: flask-service
6
7  spec:
8
9    type: NodePort
10
11    selector:
12      app: flask
13
14    ports:
15
16      - port: 5000
17        targetPort: 5000
18        nodePort: 30007
```

```
PS Z:\0m\SIT\Semester 7\DevOps Lab\assignment\devops> kubectl apply -f service.yaml
service/flask-service created
PS Z:\0m\SIT\Semester 7\DevOps Lab\assignment\devops> kubectl get services
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
flask-service       NodePort    10.109.217.59 <none>         5000:30007/TCP   9s
kubernetes           ClusterIP   10.96.0.1     <none>         443/TCP          11m
PS Z:\0m\SIT\Semester 7\DevOps Lab\assignment\devops> minikube service flask-service
```

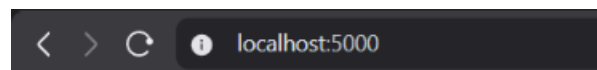
NAMESPACE	NAME	TARGET PORT	URL
default	flask-service	5000	http://192.168.49.2:30007

* Starting tunnel for service flask-service.

NAMESPACE	NAME	TARGET PORT	URL
default	flask-service		http://127.0.0.1:60022

* Opening service default/flask-service in default browser...

! Because you are using a Docker driver on windows, the terminal needs to be open to run it.



Hello World! Data stored in PostgreSQL.

3. Integrated CI/CD with Jenkins & Basic Infrastructure as Code (IaC) (3 Marks)

- **Task 3.1:** Create a Jenkins Declarative Pipeline that performs the following steps:
 - Checks out your Git repository.

- Builds the Docker image for your Flask application (from Task 1.1).
- (Optional but Recommended for higher scores): Pushes the built Docker image to a registry. (2 Marks)

```
Jenkinsfile
1 pipeline {
2   agent any
3
4   stages {
5
6     stage('Checkout') {
7       steps {
8         git 'https://github.com/ohmschrodinger/devops.git'
9       }
10    }
11
12    stage('Build Docker Image') {
13      steps {
14        bat 'docker build -t devops-flask-app .'
15      }
16    }
17
18    stage('List Docker Images') {
19      steps {
20        bat 'docker images'
21      }
22    }
23
24    // Optional
25    stage('Push Docker Image') {
26      when {
27        expression { false } // Change to true after docker login
28      }
29      steps {
30        bat 'docker push ohmschrodinger/devops-flask-app:latest'
31      }
32    }
33  }
34 }
```

Jenkins / All / New Item

New Item

Enter an item name

Select an item type

-  **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
-  **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
-  **Multibranch Pipeline**

OK

✓ #2 (28 Jul 2026, 10:29:20)



Started by user [Ohm Schrodinger](#)



Revision: de3be4e5aca939241db1a7ab45da1d370e913299

Repository: <https://github.com/ohmschrodinger/devops>

- refs/remotes/origin/main



remove dup checkout stage de3be4e

| Ohm Schrodinger at 10:29 28/07/2026

- **Task 3.2:** Install Terraform. Write a simple Terraform configuration (main.tf) to create a local file (e.g., output.txt) with some content. Initialize, plan, and apply the configuration. Show the output of terraform plan and verify the file creation. (1 Mark)

```
terraform-demo > main.tf

1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "~> 2.5"
6     }
7   }
8 }
9
10 provider "local" {}
11
12 resource "local_file" "example" {
13   filename = "output.txt"
14   content = "Hello from Terraform!"
15 }
```

Windows PowerShell

```
PS Z:\Om\SIT\Semester 7\DevOps Lab\assignment\devops\terraform-demo> terraform init
Initializing the backend...
```

Initializing provider plugins...

- Finding hashicorp/local versions matching "~> 2.5"...
- Installing hashicorp/local v2.9.0...
- Installed hashicorp/local v2.9.0 (signed by HashiCorp)

Terraform has created a lock file `.terraform.lock.hcl` to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
PS Z:\Om\SIT\Semester 7\DevOps Lab\assignment\devops\terraform-demo> terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# local_file.example will be created
+ resource "local_file" "example" {
  + content                = "Hello from Terraform!"
  + content_base64sha256   = (known after apply)
  + content_base64sha512   = (known after apply)
  + content_md5            = (known after apply)
  + content_sha1           = (known after apply)
  + content_sha256         = (known after apply)
  + content_sha512         = (known after apply)
  + directory_permission   = "0777"
  + file_permission        = "0777"
  + filename               = "output.txt"
  + id                    = (known after apply)
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

```
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.
PS Z:\Om\SIT\Semester 7\DevOps Lab\assignment\devops\terraform-demo> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# local_file.example will be created
+ resource "local_file" "example" {
  + content           = "Hello from Terraform!"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1      = (known after apply)
  + content_sha256    = (known after apply)
  + content_sha512    = (known after apply)
  + directory_permission = "0777"
  + file_permission   = "0777"
  + filename          = "output.txt"
  + id                = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.example: Creating...
local_file.example: Creation complete after 0s [id=cadbafd9572a72dc7769c97730552f5955e135fd]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS Z:\Om\SIT\Semester 7\DevOps Lab\assignment\devops\terraform-demo> type output.txt
Hello from Terraform!
PS Z:\Om\SIT\Semester 7\DevOps Lab\assignment\devops\terraform-demo>
```

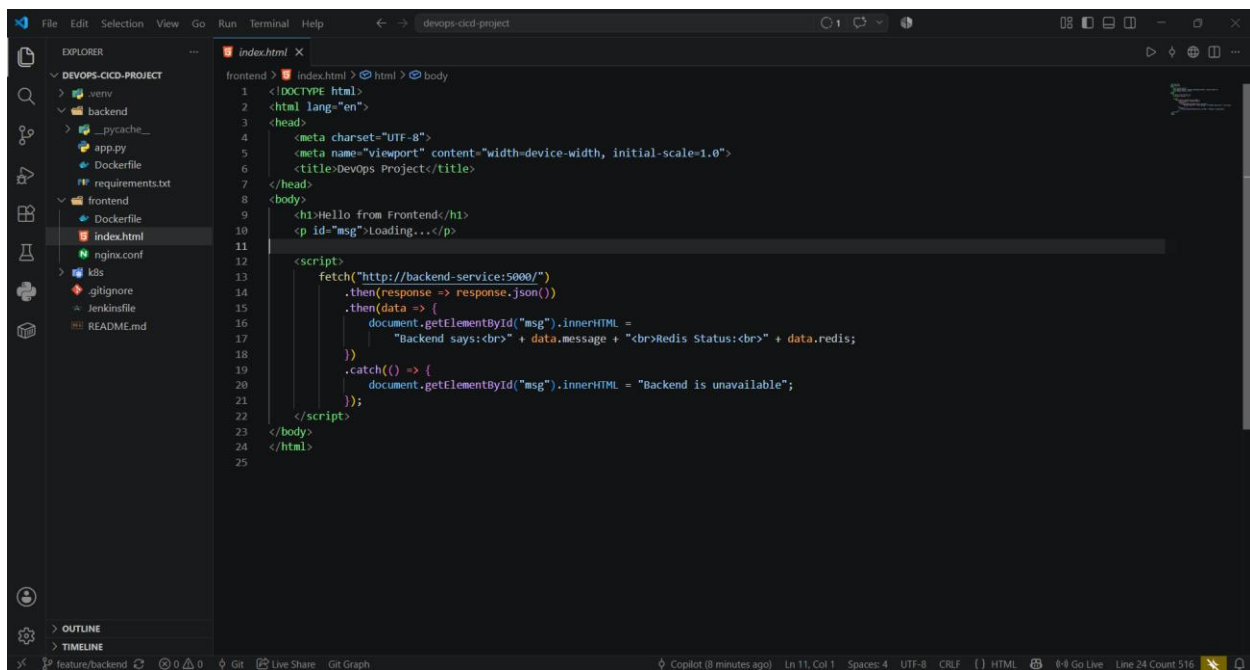
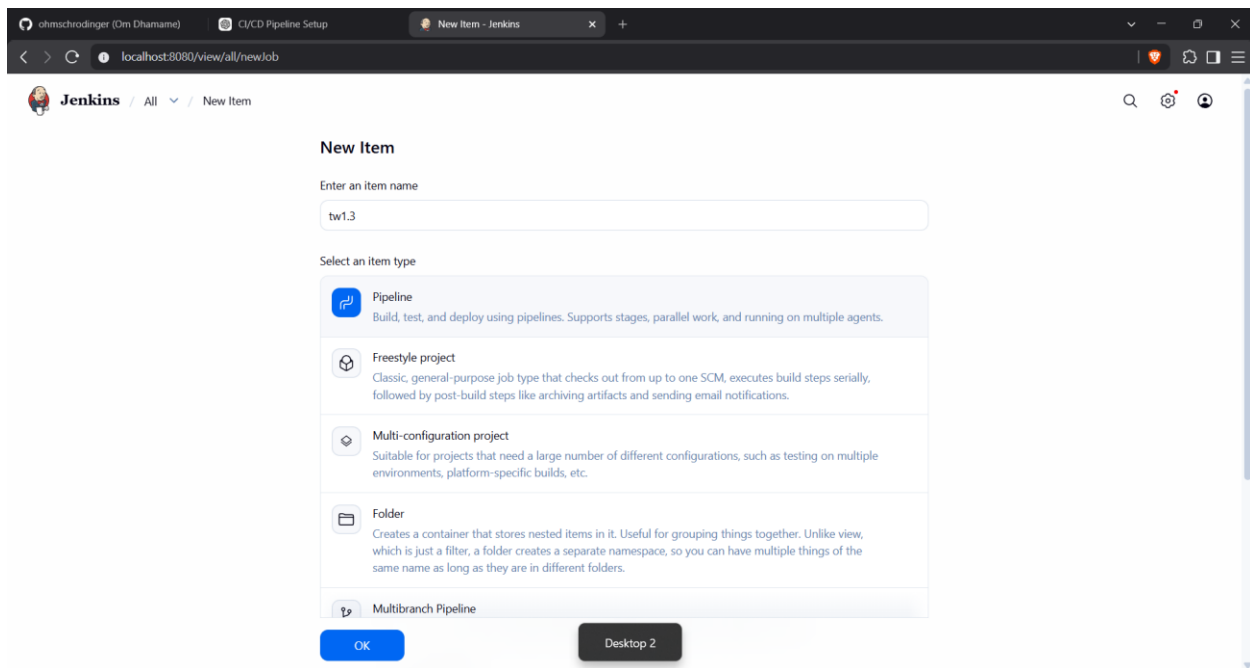
3. TW3 - ESE Lab Exam + Project Evaluation + Viva (30 Marks)

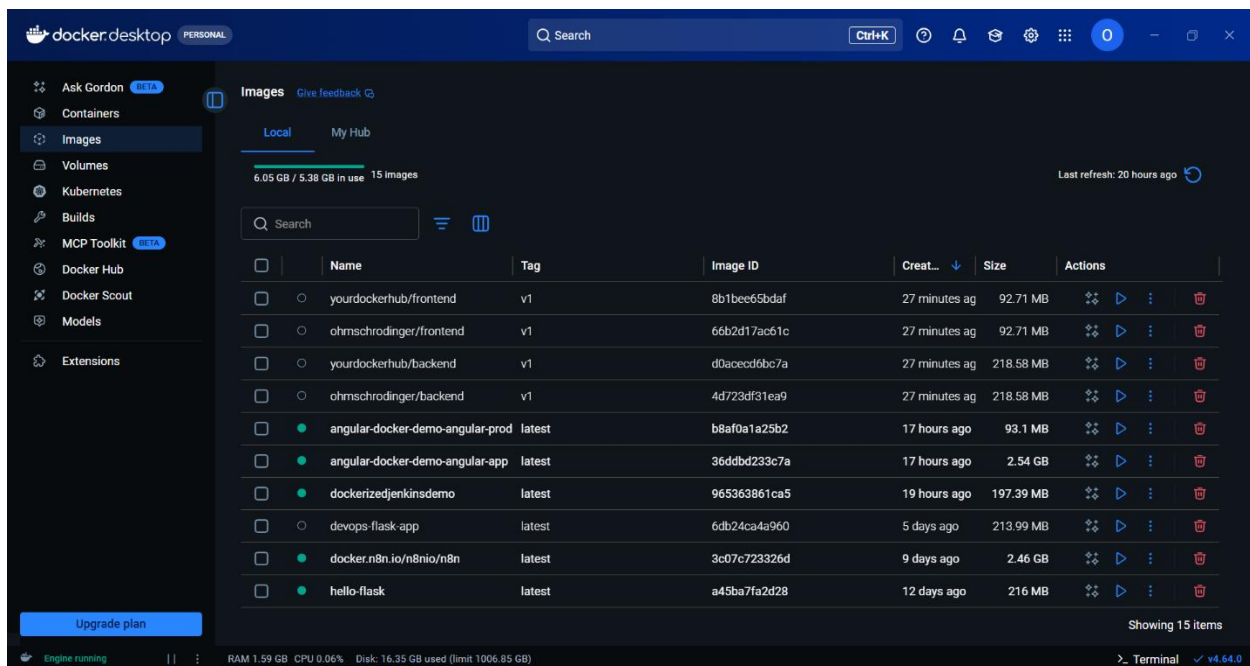
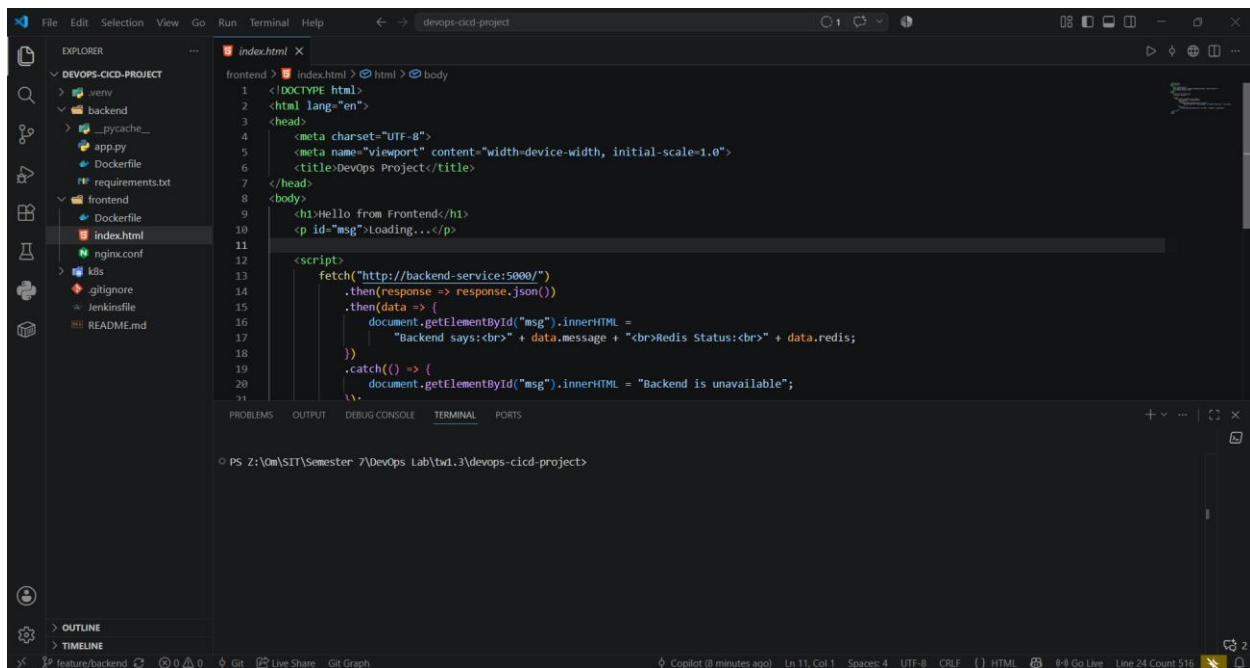
Instructions: This is a comprehensive assessment that combines practical lab work with a project implementation and a viva-voce.

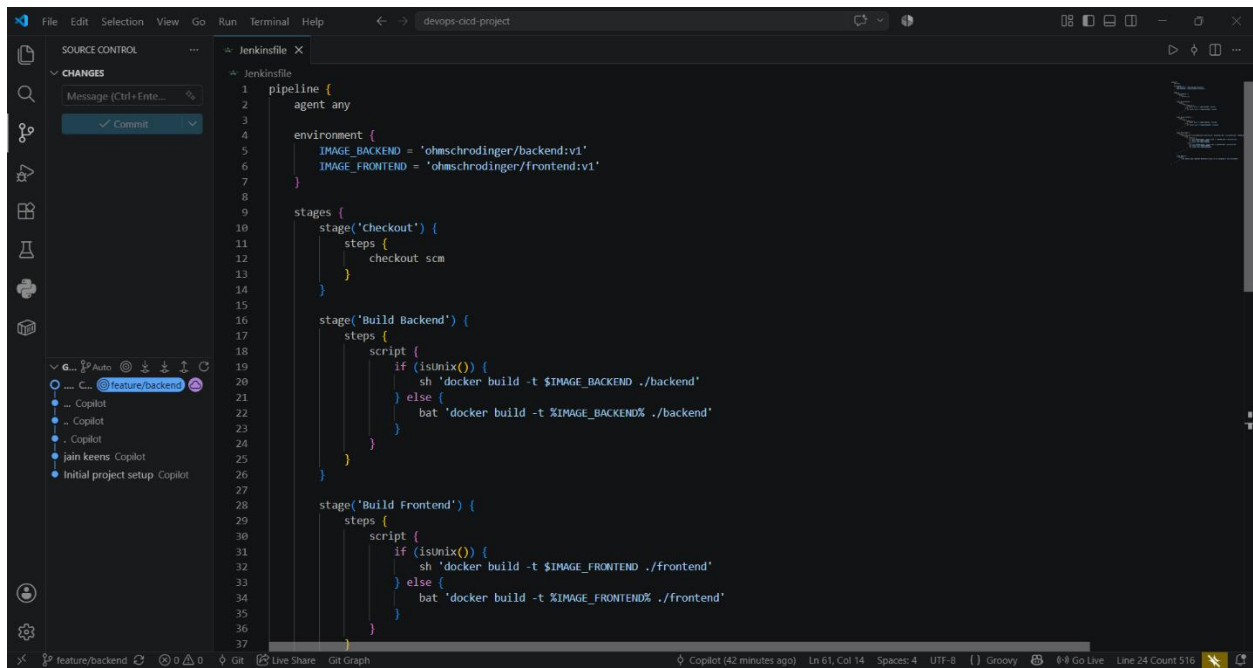
A. ESE Lab Exam (10 Marks)

Scenario: You are tasked with setting up a fully functional CI/CD pipeline for a given multi-service application (e.g., a simple microservice architecture with a frontend, backend, and database).

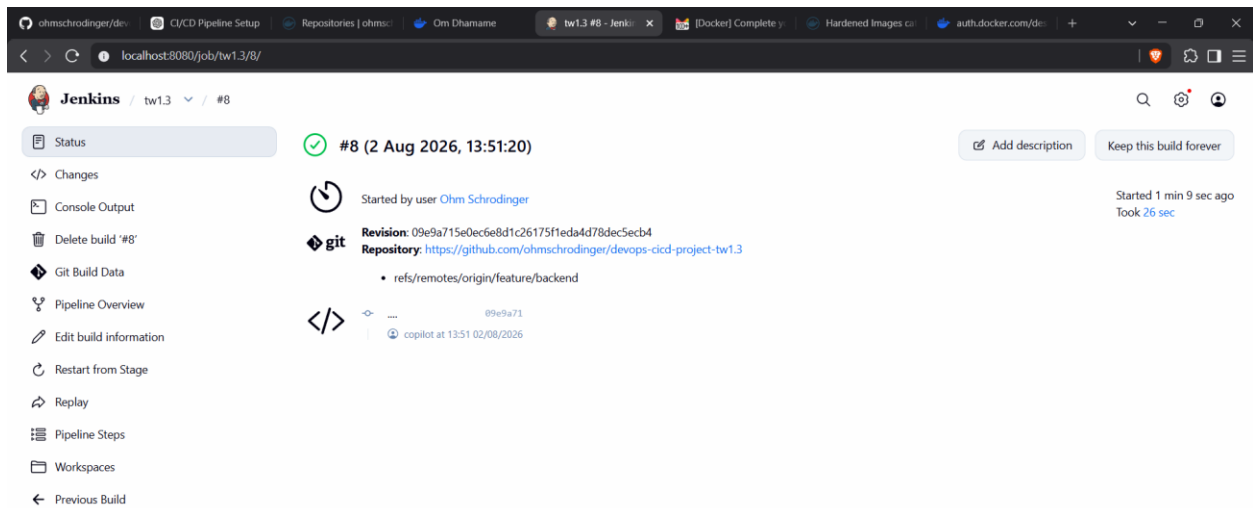
- **Task:** Given a multi-service application (frontend, backend, database components), implement a complete CI/CD pipeline from scratch.
 - Initialize a Git repository and set up a basic branching strategy.
 - Utilize Jira to create and track issues related to the deployment.
 - Develop a Jenkins pipeline that:
 - Pulls code from Git.
 - Builds Docker images for each service.
 - Pushes images to a Docker registry.
 - Deploys the services to a local Kubernetes cluster.







```
1 pipeline {
2   agent any
3
4   environment {
5     IMAGE_BACKEND = 'ohmschrodinger/backend:v1'
6     IMAGE_FRONTEND = 'ohmschrodinger/frontend:v1'
7   }
8
9   stages {
10    stage('Checkout') {
11      steps {
12        checkout scm
13      }
14    }
15
16    stage('Build Backend') {
17      steps {
18        script {
19          if (isUnix()) {
20            sh 'docker build -t $IMAGE_BACKEND ./backend'
21          } else {
22            bat 'docker build -t %IMAGE_BACKEND% ./backend'
23          }
24        }
25      }
26    }
27
28    stage('Build Frontend') {
29      steps {
30        script {
31          if (isUnix()) {
32            sh 'docker build -t $IMAGE_FRONTEND ./frontend'
33          } else {
34            bat 'docker build -t %IMAGE_FRONTEND% ./frontend'
35          }
36        }
37      }
38    }
39  }
40 }
```



Jenkins / tw1.3 / #8

Status ✓ #8 (2 Aug 2026, 13:51:20) Add description Keep this build forever

Changes

Console Output

Delete build #8

Git Build Data

Pipeline Overview

Edit build information

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Started by user Ohm Schrodinger

Revision: 09e9a715e0ec6e8d1c26175f1eda4d78dec5ecb4
Repository: <https://github.com/ohmschrodinger/devops-cicd-project-tw1.3>

- refs/remotes/origin/feature/backend

Log:

```
09e9a71
| copilot at 13:51 02/08/2026
```

Started 1 min 9 sec ago
Took 26 sec

ohmschrodinger/devops-cicd-proj... CI/CD Pipeline Setup #2 - tw1.3 - Jenkins

localhost:8080/job/tw1.3/2/stages/

Jenkins

tw1.3 / #2 / Stages

Started by Ohm Schrodinger Started 26 sec ago Took 5.7 sec on built-in

Start Checkout SCM Checkout Build Backend Build Frontend Push Images Deploy End

Deploy 80ms Started 21s ago Jenkins

Deploying to Kubernetes 15ms

Deploying to Kubernetes

Checkout SCM 1s
Checkout 99ms
Build Backend 76ms
Build Frontend 76ms
Push Images 71ms
Deploy 80ms

Jenkins 2.572

File Edit Selection View Go Run Terminal Help devops-cicd-project

Jenkinsfile M DevOps Project X 127.0.0.1:8091 README.md

http://127.0.0.1:8090/

Hello from Frontend

Backend is unavailable

EXPLORER

- DEVOPS-CICD-PROJECT
 - .venv
 - backend
 - __pycache__
 - app.py
 - Dockerfile
 - requirements.txt
 - frontend
 - Dockerfile
 - index.html
 - nginx.conf
 - k8s
 - backend-deploy...
 - frontend-deploy...
 - redis-deployment.yaml
 - services.yaml
 - .gitignore
 - Jenkinsfile
 - README.md

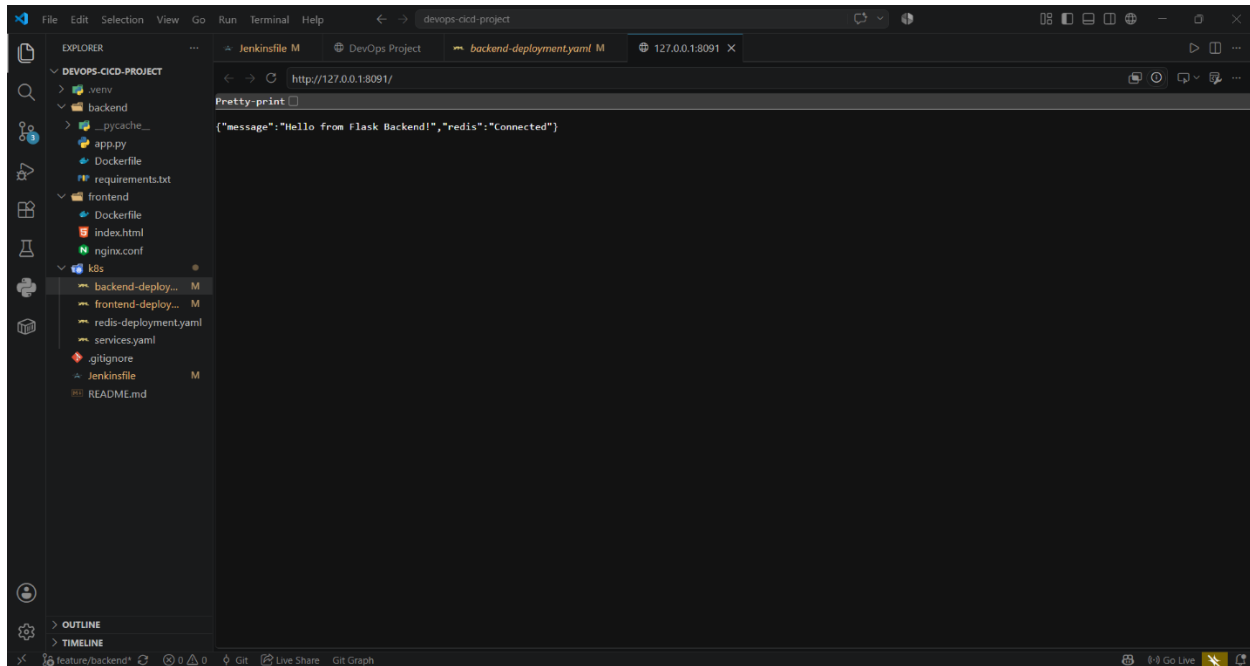
OUTLINE

TIMELINE

feature/backend*

Git Live Share Git Graph

Go Live



Evaluation Rubric (Lab Exam - 10 Marks):

- **Problem Solving & Practical Application (5 Marks):** Ability to correctly set up, configure, and integrate all required tools to achieve the functional pipeline.
- **Accuracy & Efficiency of Solution (3 Marks):** The pipeline should run smoothly without manual intervention, and the deployed application should be fully functional.
- **Troubleshooting & Debugging (2 Marks):** Ability to quickly identify and resolve any issues encountered during the setup and execution.

B. Project Evaluation (10 Marks)

Instructions: Develop a small-scale DevOps project that demonstrates your understanding and integration of at least 4-5 core DevOps tools (e.g., Git, Jira, Jenkins, Docker, Kubernetes, Terraform, Prometheus/Grafana for monitoring, Ansible for configuration management).

Project Idea (Example): Build a "Personal Blog Application" with a CI/CD pipeline.

- **Requirements:**
 - The application should be version-controlled using Git.
 - Use Jira for project and issue tracking.
 - Implement a Jenkins pipeline for continuous integration and continuous delivery.
 - Containerize the application using Docker.
 - Deploy the containerized application to a Kubernetes cluster.

- Use Terraform for provisioning at least one infrastructure component (e.g., a persistent volume for the blog data).
- (Optional for higher scores): Implement basic monitoring using Prometheus/Grafana or automated testing.

Evaluation Rubric (Project - 10 Marks):

- **Project Design & Architecture (3 Marks):** How well are the DevOps tools integrated? Is the architecture sound and scalable?
- **Functionality & Completeness (3 Marks):** Does the project work as intended? Are all specified requirements met?
- **Code Quality & Best Practices (2 Marks):** Readability, comments, adherence to best practices for Dockerfiles, Jenkinsfiles, Kubernetes YAMLs, and Terraform configurations.
- **Documentation & Presentation (2 Marks):** Clarity and completeness of project documentation (e.g., README.md, setup instructions, architecture diagrams if any) and effectiveness of the project demonstration.

C. Viva (10 Marks)

Instructions: Be prepared to answer questions related to your ESE Lab Exam and Project, as well as general DevOps concepts.

Evaluation Rubric (Viva - 10 Marks):

- **Understanding of Concepts (4 Marks):** Ability to clearly explain the theoretical concepts behind the tools used, the CI/CD pipeline, containerization, orchestration, and IaC.
- **Problem-Solving & Critical Thinking (3 Marks):** Ability to discuss challenges faced during the lab/project and how they were overcome, as well as to propose alternative solutions.
- **Clarity of Explanation & Communication (3 Marks):** Articulation and confidence in explaining technical details and answering questions.